Source Control with Git and Visual Studio

1. Introduction to Source Control

Source control, also called **version control**, is the practice of tracking and managing changes made to source code over time. A source control management system maintains a history of development so that developers can identify changes, recover previous versions, and combine work performed by multiple contributors.

Source control is useful whether you are:

- developing a small application individually;
- working on a class project with teammates;
- maintaining multiple versions of an application;
- experimenting with a new feature;
- collaborating on a larger software project.

Instead of treating a project as a collection of files that are repeatedly copied and renamed, source control records the evolution of the project.

Why Use Source Control?

Source control provides several practical benefits:

- maintains a history of changes;
- allows previous versions to be recovered;
- supports collaboration among developers;
- keeps a remote copy of a project;
- helps developers work independently before integrating their changes;
- makes it easier to identify who changed particular parts of the project.

Tip — Source Control Is More Than Backup

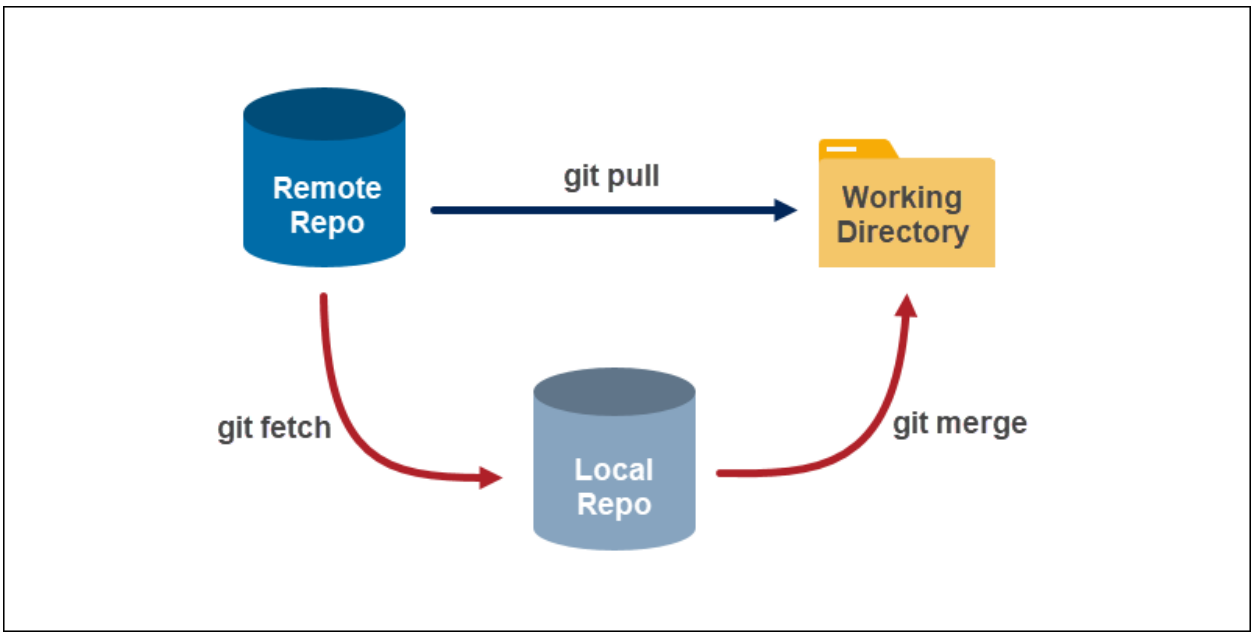
Backup protects files from being lost. Source control additionally records **how the files changed**, allows developers to compare versions, and supports collaborative development.

---

2. The Core Git Architecture

Before using Git inside Visual Studio, it is important to understand the basic architecture.

A typical Git project involves three main locations:



Working Directory

This is the project directory in which you actually edit your files.

For example:

```
StudentApp/
├── Program.cs
├── Student.cs
└── StudentApp.csproj
```

Local Repository

Git maintains the project's version history locally on your computer.

A **commit** records a version of your changes in this repository.

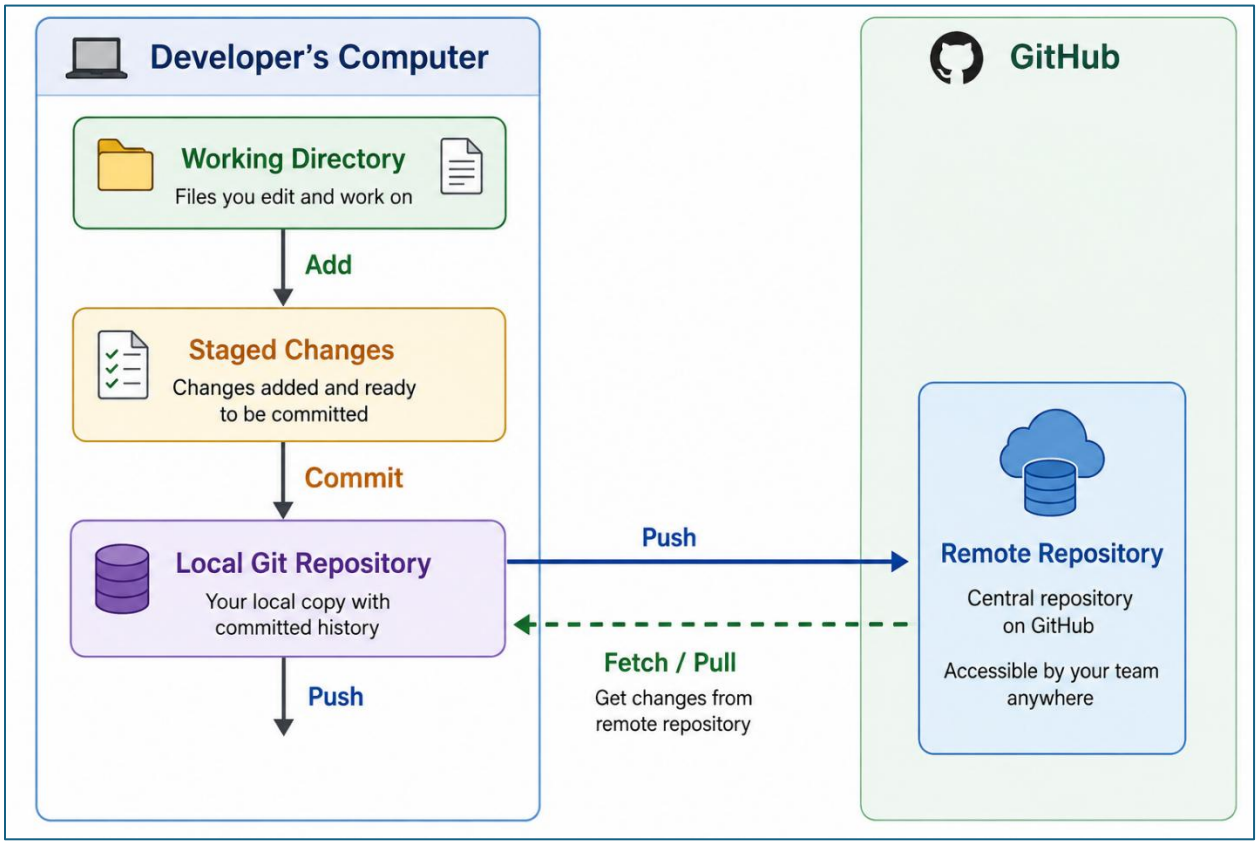
Remote Repository

A remote repository provides a shared version of the repository that can be accessed through a server or hosting service such as GitHub.

This separation is important because:

Commit and push are not the same operation.

A commit records changes in your **local repository**. A push transfers committed changes to the **remote repository**.



---

3. Repository and Git

3.1 What Is a Repository?

A **repository**, commonly shortened to **repo**, stores the project together with information about its development history.

This includes:

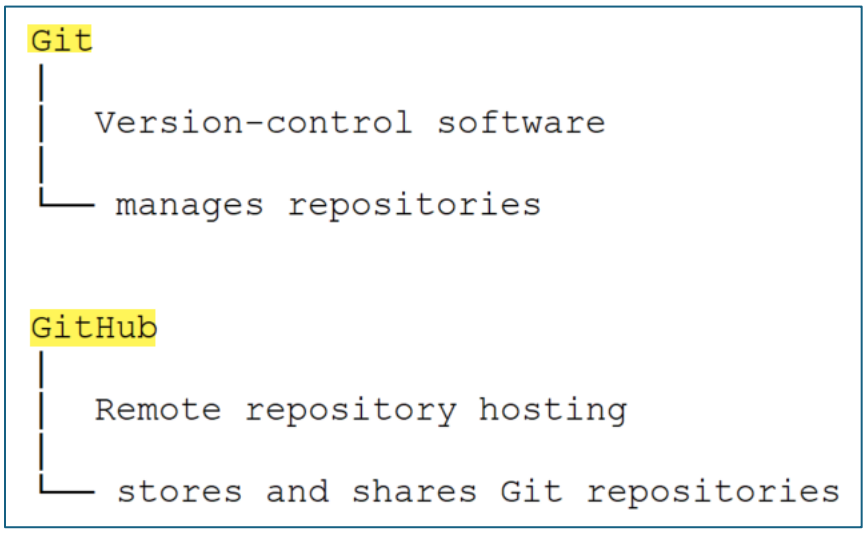
- source code;
- changes to the source code;
- revision history;
- associated metadata such as commit messages.

3.2 What Is Git?

Git is the source control system that manages repositories and their history.

Git was created by **Linus Torvalds**, who also created the Linux kernel.

A useful distinction is:



Pitfall — Git ≠ GitHub

Git is the version-control system. GitHub is a service that can host Git repositories remotely.

---

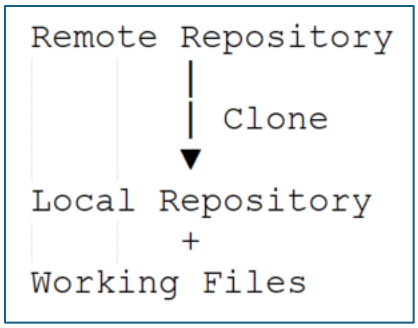
4. Essential Git Operations

The easiest way to understand Git is to follow the lifecycle of a change.

4.1 Clone

Cloning creates a local copy of an existing repository.

Conceptually:

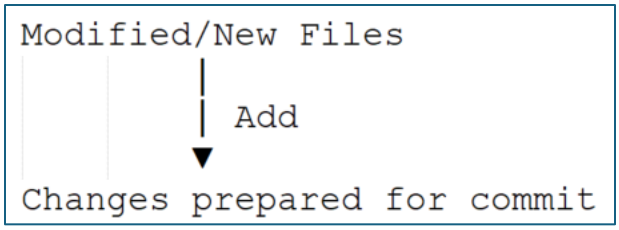


After cloning, you can edit the project locally and synchronize your work with the remote repository.

4.2 Add

When you create or modify files, Git must know which changes you intend to include in the next commit.

Conceptually:



For example, suppose you create:

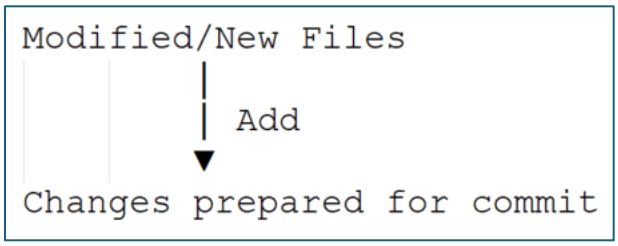
`Customer.cs`

Adding it tells Git that this file should be included in an upcoming commit.

4.3 Commit

A **commit** records your selected changes in the local repository.

Conceptually:



Each commit should contain a meaningful message describing the change.

Good:

`Add customer validation`

`Fix incorrect tax calculation`

`Implement login form`

Poor:

`changes`

`stuff`

`commit`

The commit message becomes part of the project's history.

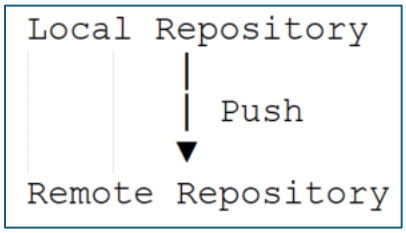
Tip — Write Meaningful Commit Messages

A useful commit message should help another developer understand **what changed** without opening every modified file.

4.4 Push

A commit initially exists only in your local repository.

Push transfers local commits to the remote repository.



The presentation's workflow explicitly distinguishes committing locally from pushing the changes to GitHub.

Example Workflow

Suppose you modify:

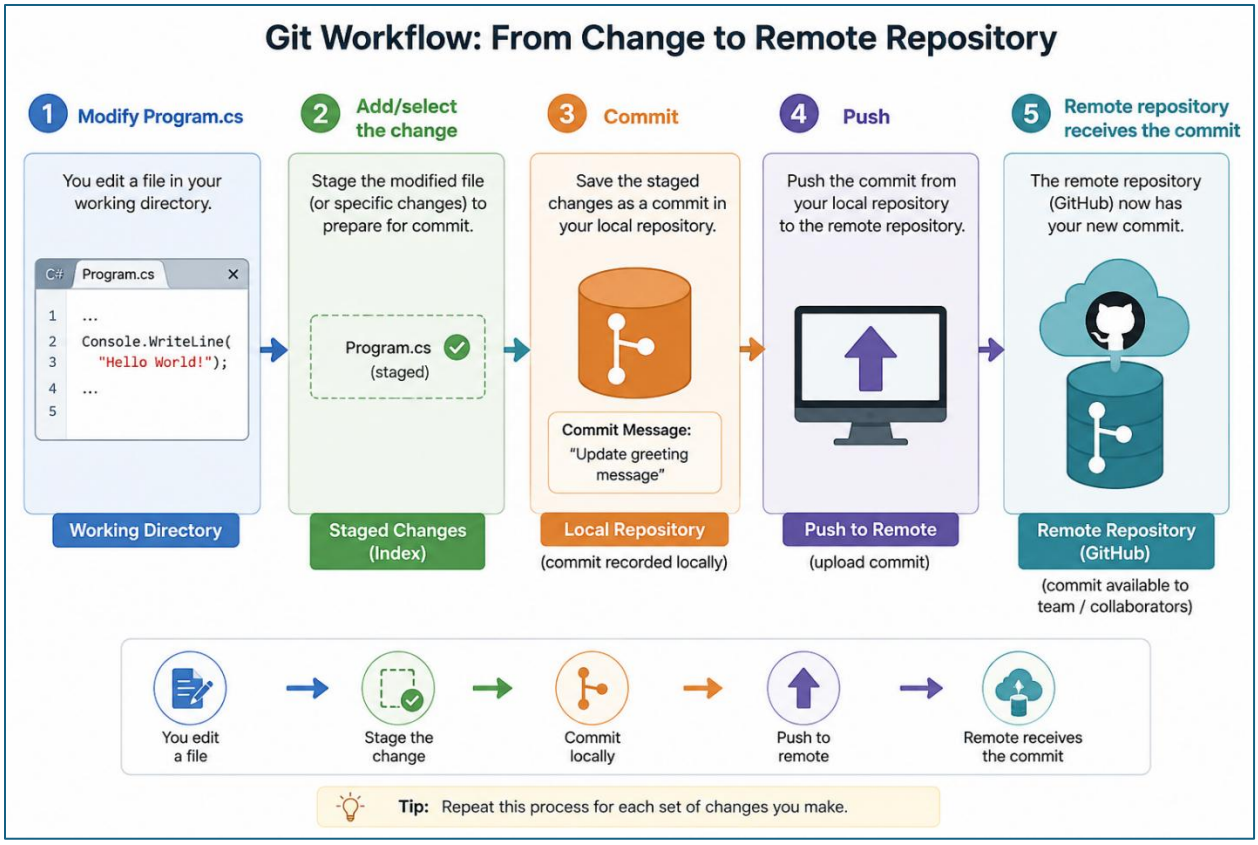
```
Console.WriteLine("Hello!");
```

to:

```
Console.WriteLine("Welcome to our application!");
```

You would conceptually perform:

```
1. Modify Program.cs
        ↓
2. Add/select the change
        ↓
3. Commit
        ↓
4. Push
        ↓
5. Remote repository receives the commit
```



---

5. Comparing and Reverting Changes

5.1 Diff

A **diff** compares versions of a file and identifies what changed.

For example:

```
- Console.WriteLine("Hello");
+ Console.WriteLine("Hello World");
```

The **–** line represents removed content and the **+** line represents added content.

Diffs are extremely useful when reviewing changes before committing or when determining what another developer modified.

5.2 Revert

Sometimes a change is incorrect or no longer needed.

Source control allows developers to return to or restore previous code states rather than manually reconstructing old code.

Note

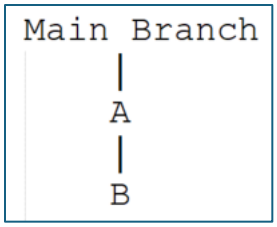
Version history is one of the major advantages of source control. Instead of maintaining files such as `ProjectFinal.cs`, `ProjectFinal2.cs`, and `ProjectReallyFinal.cs`, Git maintains the revision history systematically.

---

6. Branching

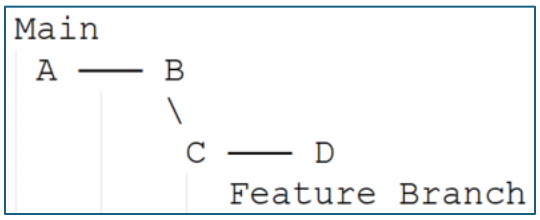
A **branch** allows development to occur separately from the main codebase.

Suppose the stable application currently contains:



You want to implement a new login feature without immediately affecting the main version.

Create a separate branch:



You can now work on the feature independently.

This is useful when:

- implementing a new feature;
- testing an experimental change;
- fixing an issue;
- allowing developers to work independently.

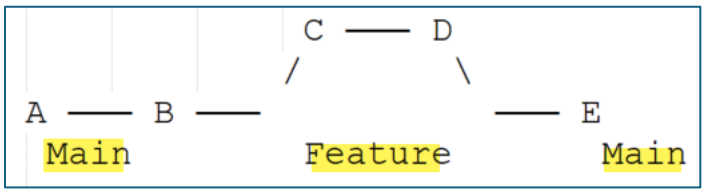
The purpose is to isolate work until it is ready to be integrated.

---

7. Merging

When work on a branch is complete, its changes can be **merged** back into another branch.

For example:



The resulting main branch contains the completed feature.

Simple Merge

If developers modified different parts of the project, Git may be able to combine the changes automatically.

Merge Conflict

A conflict can occur when developers modify overlapping code.

For example, the local version contains:

```
Console.WriteLine("Welcome!");
```

while the remote version contains:

```
Console.WriteLine("Welcome to the system!");
```

Git cannot always determine which version should be retained.

The developer must resolve the conflict.

---

8. Resolving Merge Conflicts

When a conflict occurs, Visual Studio can display the competing changes.

Conceptually:

LOCAL
```
Console.WriteLine("Welcome!");
```

REMOTE
```
Console.WriteLine("Welcome to the system!");
```

You decide which code should remain.

Possible decisions include:

Keep Local

```
Console.WriteLine("Welcome!");
```

Keep Remote

```
Console.WriteLine("Welcome to the system!");
```

Combine/Edit the Changes

```
Console.WriteLine("Welcome to our system!");
```

After resolving the conflict:

1. inspect the resulting code;
2. ensure the project still compiles;
3. test the affected functionality;
4. commit the resolved version;
5. push it to the remote repository.

The original Visual Studio workflow notes that some conflicts can be automatically merged, while others require choosing between local and remote changes. It also warns that a merge can leave code that does not compile, requiring manual correction followed by another commit and push.

Warning — A Successful Merge Does Not Guarantee Correct Code

Git combines text. It does not guarantee that the resulting application is logically correct or even compilable. Always build and test after resolving conflicts.

---

9. Fetch and Pull

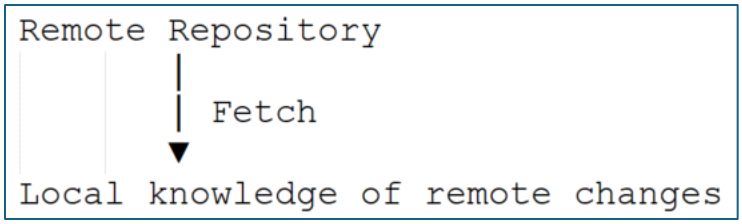
When working in a team, other developers may push changes while you are working.

You therefore need a way to retrieve their work.

Fetch

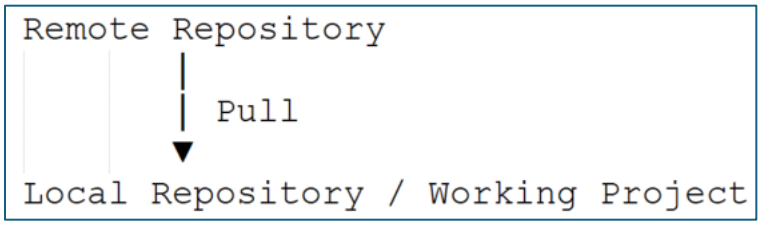
Fetch checks for and retrieves information about changes from the remote repository without directly replacing your current work.

Conceptually:



Pull

Pull retrieves remote changes and integrates them into your local work.



The Visual Studio workflow uses **Fetch** to check for team changes and **Pull** to obtain them.

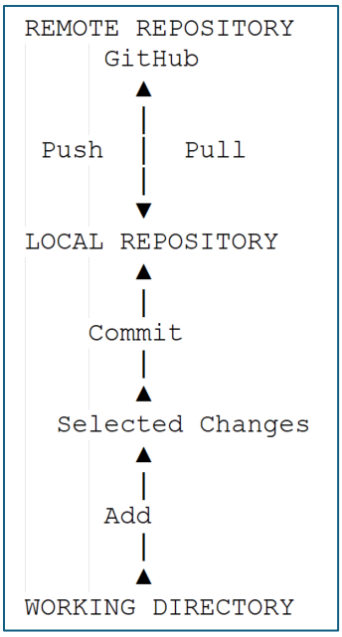
Tip

In a team project, check for remote changes regularly, particularly before beginning significant work and before pushing your own changes.

---

10. The Complete Basic Git Workflow

At this point, the major operations can be combined into one workflow.



For day-to-day development:

```
Pull
 ↓
Edit
 ↓
Review changes
 ↓
Add/select changes
 ↓
Commit
 ↓
Push
```

This is the basic cycle you should understand before working through the Visual Studio interface.

---

11. Pull Requests

A **pull request** is used to propose that changes be incorporated into another version of a repository.

Rather than directly changing the main codebase, a developer can submit changes for review.

Conceptually:

```
Developer's Changes
        ↓
     Branch
        ↓
  Pull Request
        ↓
     Review
        ↓
   Accept / Reject
        ↓
      Merge
```

Repository administrators or maintainers can review the proposed changes and decide whether they should be accepted.

Note — Pull vs. Pull Request

These terms sound similar but represent different concepts.

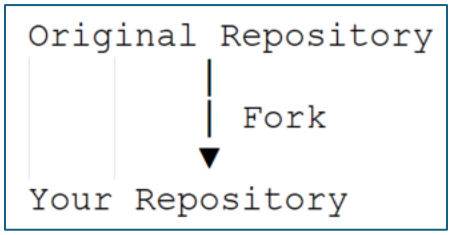
Pull: obtain changes from a remote repository.

Pull Request: propose changes for review and possible integration.

---

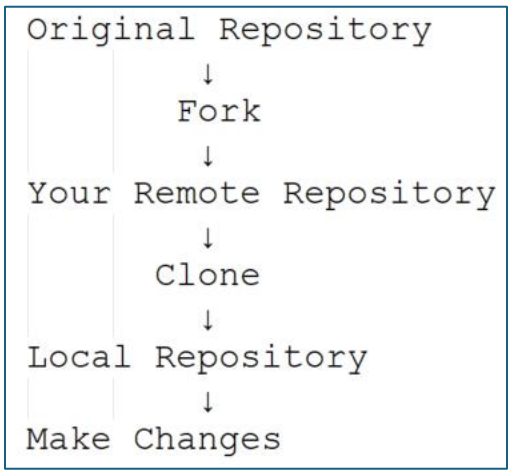
12. Forking a Repository

A **fork** creates your own copy of another repository.



You can modify your fork independently without changing the original repository.

A simplified workflow is:



Fork vs. Clone

These operations should not be confused.

Operation	Result
Clone	Creates a local copy on your computer
Fork	Creates your own remote copy of another repository

---

13. Visual Studio and Git

Visual Studio provides graphical tools for performing Git operations without requiring you to begin with the Git command-line interface.

Git itself also provides a powerful command-line interface, and third-party graphical Git clients can perform many of the same operations.

The goal here is to understand the Git concepts first and then perform those operations through Visual Studio.

Important

Visual Studio is an interface to Git. Operations such as **clone, commit, push, pull, branch, and merge** are Git concepts—not Visual Studio concepts.

---

14. Lab 1 — Clone an Existing Repository in Visual Studio

Objective

Create a local working copy of an existing remote Git repository.

Step 1 — Obtain the Repository URL

You need the Git URL for the repository you want to clone.

Step 2 — Open Visual Studio

From the Visual Studio start window, select:

Clone a repository

Step 3 — Enter the Repository URL

Paste the Git repository URL into the repository location field.

Step 4 — Select a Local Path

Choose the directory where the project should be stored locally.

For example:

```
C:\Projects\CSharpTutorial
```

Step 5 — Clone

Start the clone operation.

Visual Studio downloads the repository and creates a local working copy. The original workflow similarly specifies selecting **Clone Repository**, entering the Git URL, and choosing a local directory.

Step 6 — Inspect the Project

After cloning:

1. Open the solution;
2. Inspect the project files;
3. Build or run the application if appropriate.

You are now working with a local copy connected to the remote repository.

---

15. Repository Permissions

Cloning or viewing a repository does not automatically mean that you have permission to modify its remote version.

If another developer needs to push changes directly to a repository, that developer must have the appropriate repository permissions or be added as a contributor/collaborator.

Pitfall

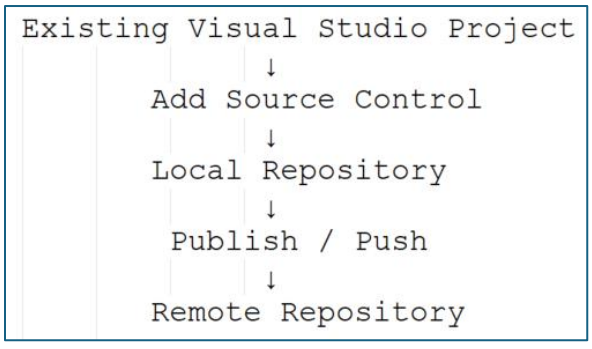
Being able to **clone** a repository does not necessarily mean that you can **push** to it.

---

16. Lab 2 — Add an Existing Visual Studio Project to Source Control

Suppose you already have a C# project on your computer and now want to manage it using Git.

The workflow is:



The original Visual Studio procedure uses the source-control/Team Explorer interface to connect the project, synchronize it, publish it to GitHub, and inspect the files included in the solution.

Step 1 — Open the Project

Open the existing solution in Visual Studio.

Step 2 — Connect Source Control

Use Visual Studio's Git/source-control interface to connect the project to Git.

Step 3 — Connect Your Account if Required

Authenticate with the account associated with the remote repository.

Step 4 — Add the Project

Add the solution to source control.

Step 5 — Publish the Repository

Publish/synchronize the repository with GitHub.

Step 6 — Check Repository Visibility

Determine whether the repository should be:

```
Public
```

or

```
Private
```

Warning

If source code is not intended for public access, ensure that repository visibility is set to **private**.

---

17. Lab 3 — Make, Commit, and Push a Change

This is the workflow you will perform repeatedly during development.

Step 1 — Modify the Project

For example, change:

```
Console.WriteLine("Hello World");
```

to:

```
Console.WriteLine("Hello Git!");
```

Save the file.

Step 2 — Review the Changes

Open Visual Studio's source-control changes view.

Confirm that the expected file appears as modified.

Do not blindly commit every changed file.

Step 3 — Review the Diff

Compare the previous and current versions.

You should see something conceptually similar to:

```
- Console.WriteLine("Hello World");
+ Console.WriteLine("Hello Git!");
```

Confirm that the change is intentional.

Step 4 — Enter a Meaningful Commit Message

For example:

```
Update console greeting
```

Avoid:

```
commit changes
```

The Visual Studio procedure explicitly recommends a meaningful commit message rather than a generic message such as "commit changes."

Step 5 — Commit

Commit the selected changes.

At this point:

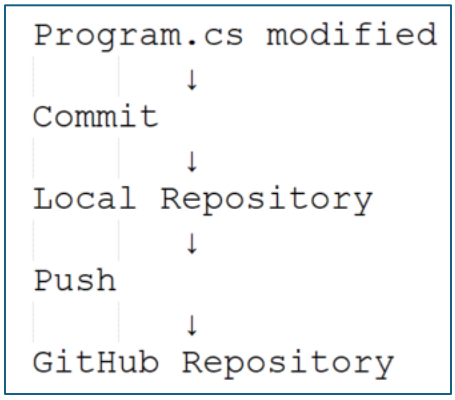
✓ Local repository updated

✗ Remote repository not necessarily updated

Step 6 — Push

Push the commit to the remote repository.

The workflow then becomes:



The Visual Studio workflow concludes by synchronizing and pushing the changes to the server.

---

18. Lab 4 — Get Changes Made by a Teammate

Suppose another team member changes the project and pushes the change.

Your computer does not automatically contain that modification.

Step 1 — Check for Remote Changes

Use **Fetch** to check for changes.

Step 2 — Inspect Available Changes

Determine whether the remote repository contains commits that you do not have locally.

Step 3 — Pull

Use **Pull** to retrieve and integrate the changes.

Step 4 — Inspect the Project

Check the changed files.

Step 5 — Build the Project

Verify that the updated application still compiles and works correctly.

---

19. Lab 5 — Work with a Branch

Suppose you need to implement a new feature:

Add student grade calculation.

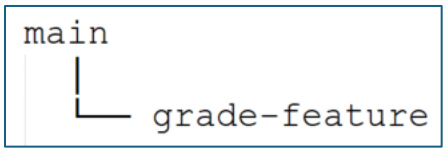
Rather than immediately changing the main branch:

Step 1 — Start from the Current Main Branch

Ensure that your local code is current.

Step 2 — Create a Branch

Conceptually:



Step 3 — Make the Changes

Implement the feature on the branch.

For example:

```
public double CalculateAverage(int[] grades)
{
    int total = 0;

    foreach (int grade in grades)
    {
        total += grade;
    }

    return (double)total / grades.Length;
}
```

Step 4 — Test the Feature

Confirm that the code compiles and behaves as expected.

Step 5 — Commit the Changes

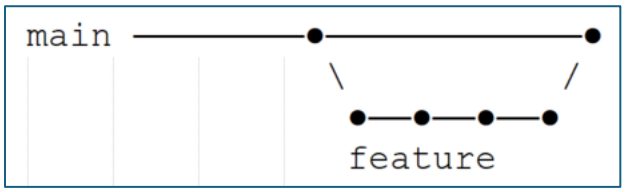
Use a meaningful message:

```
Implement grade average calculation
```

Step 6 — Merge When Complete

Once the branch is ready, merge it into the main codebase.

The key idea is:



The branch isolates development; the merge integrates completed work.

---

20. Lab 6 — Resolve a Merge Conflict

Consider two developers editing the same statement.

Original

```
Console.WriteLine("Student System");
```

Developer A

```
Console.WriteLine("Student Management System");
```

Developer B

```
Console.WriteLine("Welcome to the Student System");
```

When these changes are combined, Git may report a conflict.

Resolution Procedure

Step 1: Open the conflicted file.

Step 2: Compare the local and remote versions.

Step 3: Decide which code should remain.

You might manually combine them:

```
Console.WriteLine("Welcome to the Student Management System");
```

Step 4: Save the resolved file.

Step 5: Build the application.

Step 6: Test the affected functionality.

Step 7: Commit the merge resolution.

Step 8: Push the corrected version.

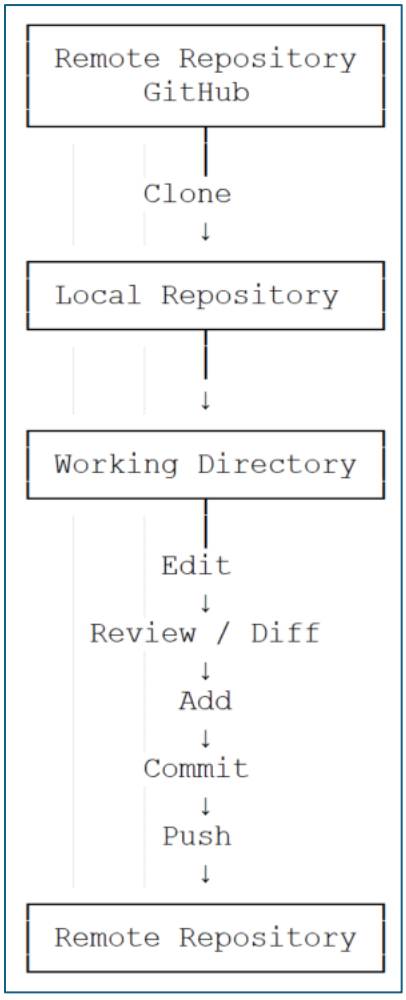
Warning

Never resolve a conflict by automatically selecting **Local** or **Remote** without first understanding what each version contains.

---

21. Putting Everything Together

The complete workflow introduced in this section can now be represented as:



In collaborative development, add:

```
Branch → Develop → Commit → Merge
```

and regularly:

```
Fetch / Pull
```

to obtain changes made by teammates.

---

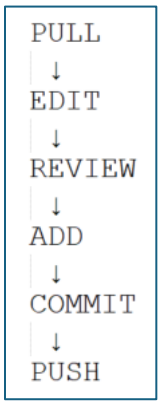
22. Key Terminology

Term	Meaning
Source Control	Systematic management of changes to source code
Repository	Project and its version history
Git	Version-control system used to manage repositories
Clone	Create a local copy of an existing repository
Add	Prepare new/modified files for a commit
Commit	Record changes in the local repository
Push	Send local commits to a remote repository
Fetch	Check/retrieve information about remote changes
Pull	Retrieve and integrate remote changes
Diff	Compare versions of files
Revert	Return code toward an earlier version/state
Branch	Independent line of development
Merge	Combine changes from different development lines
Merge Conflict	Situation requiring manual reconciliation of competing changes
Pull Request	Proposal to review and integrate changes
Fork	Create your own remote copy of another repository

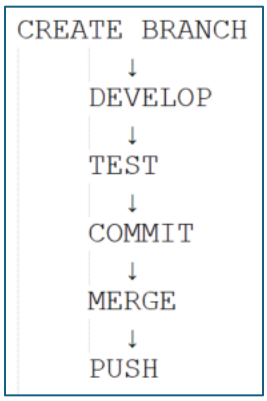
---

23. Practical Workflow to Remember

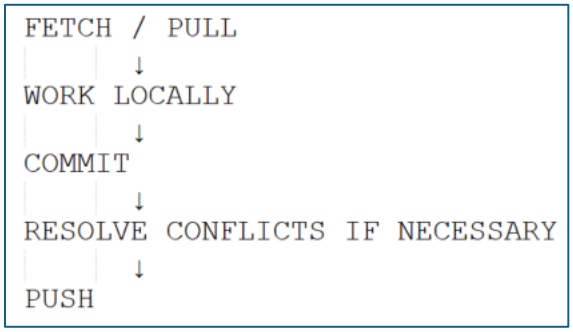
For normal development:



For feature development:



For team collaboration:



The most important conceptual distinction is to understand **where each operation acts**:

`Working Directory → Commit → Local Repository → Push → Remote Repository`

Once this mental model is clear, the corresponding Visual Studio operations become much easier to understand and use.